

UNIVERSIDADE FEDERAL DE ALAGOAS

Instituto de Computação

Curso de Engenharia da Computação

Larissa Ferreira Dias de Souza
Otávio Joshua Costa Brandão Menezes
Zilderlan Naty dos Santos

UNDERWATER GAME

Repositório: [Acessar no GitHub](#)

Jogo: [Jogar no navegador](#)

Vídeo: [Assistir apresentação](#)

Larissa Ferreira Dias de Souza
Otávio Joshua Costa Brandão Menezes
Zilderlan Naty dos Santos

UNDERWATER GAME

Relatório de Projeto apresentado à disciplina de Engenharia de Software do curso de Engenharia da Computação da Universidade Federal de Alagoas, como requisito parcial para aprovação na disciplina.

Docente: Prof. Dr. Arturo Hernandez Dominguez

Universidade Federal de Alagoas – UFAL
Instituto de Computação
Maceió – AL
2026

Sumário

1	ENUNCIADO DO PROJETO	4
2	METODOLOGIA	5
3	TÉCNICAS DE REUSO DE SOFTWARE UTILIZADAS	6
3.1	Microserviços	6
3.2	Componentes de software	6
4	DESENVOLVIMENTO DO PROJETO	8
4.1	Desenvolvimento do Projeto segundo a Metodologia Ágil e as Técnicas de Reuso	8
4.1.1	Sprint 1 — Infraestrutura e Backend Base	8
4.1.2	Sprint 2 — Multiplayer e Mecânicas Principais	9
4.1.3	Sprint 3 — Polimento, Autenticação e Produção	10
4.1.4	Relação entre Scrum e Reuso de Software	11
5	PRODUCT BACKLOG E SPRINT BACKLOG	12
5.1	Product Backlog	12
5.2	Sprint Backlog – Sprint 1	14
5.3	Sprint Backlog – Sprint 2	14
5.4	Sprint Backlog – Sprint 3	15
6	ARQUITETURA DO SISTEMA COM MICROSERVIÇOS	17
6.1	Arquitetura do sistema utilizando microserviços	17
6.2	API Gateway	17
6.3	Especificação dos microserviços e componentes de software	18
6.3.1	Microserviço 1: [Auth Service — porta 3004]	18
6.3.2	Microserviço 2: [Game Service — porta 3001]	19
6.3.3	Microserviço 3: [Puzzle API — porta 3002]	20
6.3.4	Microserviço 4: [Score Service — porta 3003]	21
6.3.5	Componentes de software: Frontend	23
6.3.6	Componentes de software: Backend	25
6.4	Sistema de recomendação	29
6.5	Microserviços em execução	30
7	TECNOLOGIAS UTILIZADAS	31

8	TELAS DO SISTEMA	33
8.1	Tela de login	33
8.2	Esperando por um segundo jogador	33
8.3	Cenário principal do jogo	34
8.4	Interface do puzzle	35
8.5	Interface de resultados da partida	36
9	REPOSITÓRIO NO GITHUB	38
9.1	Instruções para execução do sistema	38
9.1.1	Pré-requisitos	38
9.1.2	Execução com Docker Compose	38
9.1.3	Serviços executados localmente	39
9.1.4	Execução manual sem Docker	39
9.1.5	Observações sobre execução	40
10	VÍDEO DO SISTEMA	42
11	REFERÊNCIAS BIBLIOGRÁFICAS	43

1 Enunciado do projeto

O Underwater Game é um jogo digital online não violento, desenvolvido como projeto da disciplina de Engenharia de Software, com foco na aplicação de arquitetura de microserviços e componentes de software. A proposta do sistema é oferecer uma experiência multiplayer cooperativa para dois jogadores, em que cada participante controla um submarino em um ambiente subaquático, explorando o mapa, interagindo com animais marinhos e resolvendo desafios educativos em formato de minigame.

O jogo é executado em ambiente web e possui frontend desenvolvido com Phaser e TypeScript. A interface é responsável por renderizar o mapa, os submarinos, os animais, o HUD, os puzzles e a tela final da partida. Já as regras principais do jogo são distribuídas entre microserviços, garantindo separação de responsabilidades entre autenticação, estado da partida, puzzles e pontuação. Durante a partida, dois jogadores controlam submarinos em uma mesma sala online, explorando um ambiente subaquático compartilhado e sincronizado em tempo real. Ao encontrar animais marinhos, os jogadores podem iniciar puzzles educativos no estilo “força submarina”, utilizando dicas para descobrir o nome do animal.

O jogo possui mecânicas de oxigênio, vidas, colisões, respawn e pontuação final. A partida pode terminar pela descoberta de todos os animais ou pela eliminação de um jogador. Ao final, o sistema apresenta o vencedor, o placar e um resumo do desempenho dos participantes. A arquitetura foi organizada em um monorepo com frontend e microserviços independentes. O auth-service cuida do login por nickname, o game-service gerencia a partida em tempo real, o puzzle-api fornece os dados dos desafios e o score-service calcula o resultado final. Dessa forma, o Underwater Game aplica, na prática, conceitos de microserviços, componentes de software, comunicação em tempo real e separação de responsabilidades.

2 Metodologia

A metodologia ágil utilizada no desenvolvimento do projeto foi o Scrum, adaptado ao contexto acadêmico e ao tamanho da equipe. O desenvolvimento foi organizado em ciclos incrementais, nos quais as funcionalidades eram planejadas, implementadas, testadas e integradas gradualmente ao sistema.

Inicialmente, a equipe definiu o Product Backlog, contendo as principais funcionalidades esperadas para o jogo, como a criação da arquitetura de microserviços, o multiplayer online, a geração do mapa, os puzzles, o sistema de oxigênio e vidas, a pontuação final, a autenticação por nickname e as telas de interface. A partir desse backlog geral, as tarefas foram distribuídas em sprints, priorizando primeiro a base funcional do sistema e depois os recursos de gameplay, integração e polimento.

Durante o desenvolvimento, cada sprint foi organizada em tarefas menores, geralmente associadas a issues no GitHub. Para cada funcionalidade, eram criadas branches específicas, permitindo que os integrantes trabalhassem de forma separada e depois integrassem as alterações por meio de pull requests. Esse fluxo ajudou a manter o controle das mudanças, facilitar a revisão do código e reduzir conflitos entre as partes do projeto.

O Scrum também foi aplicado por meio de acompanhamento contínuo do progresso da equipe. Ao final de cada etapa, as funcionalidades implementadas eram testadas, avaliadas e, quando necessário, corrigidas em novas issues. Dessa forma, o projeto evoluiu de maneira incremental: primeiro com a infraestrutura e os microserviços principais, depois com o gameplay multiplayer e, por fim, com autenticação, pontuação, telas finais, efeitos visuais e correções de bugs. Portanto, o uso do Scrum contribuiu para organizar o desenvolvimento do Underwater Game, permitindo dividir um sistema complexo em entregas menores, acompanhar o progresso da equipe e integrar gradualmente os componentes de software e microserviços.

3 Técnicas de reuso de software utilizadas

O projeto utiliza duas técnicas principais de reuso de software: microserviços e componentes de software. Essas abordagens foram escolhidas para separar responsabilidades, facilitar a manutenção do código e permitir que partes do sistema possam ser reaproveitadas ou modificadas sem afetar toda a aplicação.

3.1 Microserviços

No nível da arquitetura, o projeto foi dividido em microserviços independentes, cada um responsável por uma parte específica do sistema. Essa separação permite que cada serviço tenha uma função bem definida, podendo ser mantido, testado, substituído ou expandido de forma isolada.

O auth-service, por exemplo, concentra a autenticação por nickname e geração de token. Essa lógica não fica acoplada ao jogo em si, podendo ser reutilizada por qualquer outra aplicação que precise de entrada simples por apelido. O game-service centraliza a lógica principal da partida, como sala multiplayer, estado dos jogadores, eventos em tempo real, oxigênio, vidas e fim de jogo. Já o puzzle-api funciona como um serviço especializado em fornecer e validar os desafios relacionados aos animais marinhos. Por fim, o score-service calcula a pontuação final da partida, mantendo a regra de pontuação fora do frontend e do motor principal do jogo.

Com essa organização, o sistema evita concentrar todas as responsabilidades em um único bloco de código. Cada microserviço possui uma responsabilidade própria e se comunica com os demais por HTTP REST ou Socket.IO. Isso melhora a modularidade do projeto e facilita a evolução futura, pois novas funcionalidades podem ser adicionadas sem exigir grandes alterações nos serviços já existentes.

3.2 Componentes de software

Além dos microserviços, o projeto também aplica reuso por meio de componentes de software no frontend. Como o jogo foi desenvolvido com Phaser e TypeScript, várias responsabilidades visuais e lógicas foram separadas em cenas, entidades e sistemas reutilizáveis.

As cenas principais, como NicknameScene, MenuScene, GameScene, PuzzleScene e EndScene, organizam cada etapa do fluxo do jogo. Cada uma possui uma função específica:

login, espera por jogadores, partida principal, minigame e tela de resultado. Isso evita que toda a interface fique concentrada em uma única classe.

Também foram criados componentes e sistemas internos reutilizáveis. O `SocketManager` centraliza a comunicação com o `game-service`, enquanto o `ScoreSocketManager` cuida da comunicação com o `score-service`. O `MapGenerationSystem` é responsável pela geração determinística do mapa por seed, podendo ser reutilizado sempre que uma nova partida começa. O `MovementSystem` separa a lógica de movimentação do submarino da cena principal. O HUD é utilizado para exibir oxigênio, vidas e nomes dos jogadores, sendo instanciado tanto para o jogador local quanto para o parceiro. O `WaterEffectsSystem` concentra efeitos visuais, como bolhas e vinheta, sem interferir nas regras do jogo.

Essa divisão em componentes torna o frontend mais organizado e facilita a manutenção. Quando uma funcionalidade precisa ser ajustada, como movimentação, HUD, mapa ou comunicação com o backend, a alteração pode ser feita no componente correspondente sem modificar toda a `GameScene`. Dessa forma, o projeto aplica o reuso não apenas na arquitetura geral com microserviços, mas também na estrutura interna do frontend.

4 Desenvolvimento do projeto

4.1 Desenvolvimento do Projeto segundo a Metodologia Ágil e as Técnicas de Reuso

O desenvolvimento do *Underwater Game* seguiu um fluxo incremental baseado em Scrum, organizado em três sprints principais. Cada sprint foi composta por issues no GitHub, utilizadas como unidades de trabalho menores e associadas às entregas planejadas para cada etapa do projeto.

Além da metodologia ágil, o projeto aplicou técnicas de reuso de software por meio da divisão em microserviços e componentes internos do frontend. Os microserviços permitiram separar responsabilidades como autenticação, lógica da partida, puzzles e pontuação. No frontend, componentes como `SocketManager`, `MovementSystem`, `MapGenerationSystem`, `HUD` e `WaterEffectsSystem` facilitaram a organização, manutenção e reutilização de partes do sistema.

4.1.1 Sprint 1 — Infraestrutura e Backend Base

A primeira sprint teve como foco a criação da base do projeto. Nessa etapa, foram definidos o monorepo, a estrutura inicial dos microserviços, a comunicação básica em tempo real e o deploy inicial. O principal entregável foi permitir que dois navegadores conseguissem abrir o jogo e se conectar ao mesmo servidor.

Issue	Título	Descrição
#1	Fundação e esqueleto — Backend	Criação da estrutura de pastas do monorepo, configuração do Docker Compose com os serviços e definição dos contratos Socket.IO iniciais.
#2	API REST de animais — Backend	Implementação da <code>puzzle-api</code> com catálogo de cinco animais marinhos, além dos endpoints para adivinhar letras e solicitar dicas.
#3	Tela principal com canvas — Frontend	Criação do projeto Vite com Phaser, configuração das cenas iniciais e implementação da <code>GameScene</code> base.
#4	Submarino seguindo o cursor — Frontend	Implementação do <code>MovementSystem</code> , com movimentação suavizada, e criação do <code>PlayerSubmarine</code> como entidade visual do jogador.
#5	Mapa base com tiles e obstáculos — Frontend	Implementação do <code>MapGenerationSystem</code> , responsável pela geração determinística do mapa por seed, obstáculos estáticos e decorações.
#6	Deploy contínuo do backend — DevOps	Publicação dos serviços no Render, configuração de variáveis de ambiente, CORS e documentação inicial de deploy.

Tabela 1 – Sprint 1 — Infraestrutura e Backend Base

4.1.2 Sprint 2 — Multiplayer e Mecânicas Principais

A segunda sprint teve como foco a sincronização em tempo real e a implementação das principais mecânicas do jogo. Ao final dessa etapa, já era possível executar uma partida funcional com dois jogadores, movimentação dos submarinos, interação com animais, abertura de puzzles e atualização do estado da partida.

Issue	Título	Descrição
#7	Sincronizar submarinos via Socket.IO	Implementação dos eventos <code>player:move</code> e <code>player:moved</code> , permitindo sincronizar a posição dos submarinos em tempo real.
#8	HUD com O ₂ e corações	Criação do componente HUD, reutilizado para exibir oxigênio, vidas e informações dos jogadores.
#9	Proximidade com animais e abertura do puzzle	Implementação da detecção de aproximação dos animais, envio do evento <code>animal:approach</code> e abertura do puzzle pelo backend.
#10	Minigame da força na PuzzleScene	Criação da tela de puzzle com letras, input de teclado, feedback de erro, botão de dica e fechamento ao concluir o desafio.
#11	O ₂ , dano e derrota	Implementação da lógica de oxigênio, colisão com obstáculos, perda de vidas, respawn individual e eliminação definitiva.
#12	Teste de integração ponta a ponta	Validação manual do fluxo completo da partida, desde a entrada dos jogadores até a resolução dos puzzles e encerramento.

Tabela 2 – Sprint 2 — Multiplayer e Mecânicas Principais

4.1.3 Sprint 3 — Polimento, Autenticação e Produção

A terceira sprint teve como foco o polimento da experiência do usuário, autenticação, telas finais, pontuação e ajustes de produção. Nessa etapa, o jogo recebeu melhorias visuais, tela de resultado, sistema de login por apelido e correções relacionadas ao fluxo de reinício e conexão dos jogadores.

Issue	Título	Descrição
#13	Substituição de placeholders por sprites PNG	Integração de assets visuais via <code>assetsMap.ts</code> , incluindo submarinos, animais, corações e bolha de oxigênio.
#14	Telas de game over, vitória e pontuação final	Implementação da <code>EndScene</code> , com tabela de pontuação por animal, resumo dos jogadores e botão de jogar novamente.
#15	Efeitos visuais de água	Criação do <code>WaterEffectsSystem</code> , adicionando bolhas animadas no mapa e vinheta fixa na câmera.
#16	Correções de desconexão e reconexão	Ajustes no fluxo de desconexão, tratamento de perda de conexão e retorno do jogador ao menu quando necessário.
#35	Login por apelido antes da partida	Implementação do <code>auth-service</code> , da <code>NicknameScene</code> , validação de token no <code>game-service</code> e exibição dos nicknames no HUD.

Tabela 3 – Sprint 3 — Polimento, Autenticação e Produção

4.1.4 Relação entre Scrum e Reuso de Software

A organização em sprints facilitou a aplicação das técnicas de reuso no projeto. Em vez de desenvolver o jogo como uma aplicação única e fortemente acoplada, a equipe dividiu o sistema em microserviços e componentes reutilizáveis.

No backend, cada microserviço possui uma responsabilidade específica. O `auth-service` trata autenticação, o `game-service` gerencia o estado da partida, o `puzzle-api` fornece os desafios dos animais e o `score-service` calcula a pontuação final. Essa divisão permite que cada serviço seja modificado, testado ou substituído de forma independente.

No frontend, a separação em componentes também contribuiu para o reuso. A `GameScene` coordena a partida, mas responsabilidades específicas foram extraídas para sistemas próprios, como `MapGenerationSystem`, `SocketManager`, `HUD` e `WaterEffectsSystem`. Com isso, o código se tornou mais organizado, modular e fácil de manter. Dessa forma, o uso do Scrum ajudou a construir o projeto de forma incremental, enquanto as técnicas de reuso permitiram manter uma arquitetura mais limpa, modular e preparada para evolução.

5 Product Backlog e Sprint Backlog

5.1 Product Backlog

O Product Backlog do projeto foi organizado a partir das principais funcionalidades necessárias para a construção do *Underwater Game*. As histórias foram distribuídas em três sprints, seguindo uma evolução incremental: infraestrutura base, mecânicas principais e polimento para produção.

ID	História de Usuário	Prioridade	Sprint
US01	Como equipe de desenvolvimento, queremos criar a fundação e o esqueleto do projeto para organizar o monorepo e os serviços iniciais.	Muito Alta	1
US02	Como jogador, quero que o jogo tenha uma API de animais com dados dos 5 animais da demo, para que os puzzles possam usar esse catálogo.	Muito Alta	1
US03	Como jogador, quero acessar uma tela principal com canvas e cena no Phaser, para visualizar o ambiente do jogo.	Alta	1
US04	Como jogador, quero controlar um submarino seguindo o cursor do mouse com suavização, para ter uma movimentação fluida.	Alta	1
US05	Como jogador, quero explorar um mapa base com tiles, profundidade e obstáculos estáticos, para ter um ambiente navegável.	Alta	1
US06	Como equipe de desenvolvimento, queremos configurar o deploy contínuo do backend, para testar os serviços em ambiente online.	Média	1
US07	Como jogador, quero ver a posição dos submarinos sincronizada em tempo real, para jogar online com outro participante.	Alta	2

ID	História de Usuário	Prioridade	Sprint
US08	Como jogador, quero visualizar um HUD com barra de O ₂ e corações de vida, para acompanhar meu estado durante a partida.	Alta	2
US09	Como jogador, quero que o jogo detecte a proximidade com animais e abra um puzzle, para interagir com o ambiente subaquático.	Alta	2
US10	Como jogador, quero resolver um mini-game da força na PuzzleScene , para descobrir os animais marinhos.	Alta	2
US11	Como jogador, quero que o jogo tenha lógica de O ₂ , dano por obstáculo e condições de derrota, para criar desafio durante a exploração.	Alta	2
US12	Como equipe de desenvolvimento, queremos realizar um teste de integração ponta a ponta, para validar uma partida completa.	Média	2
US13	Como jogador, quero que os placeholders sejam substituídos por sprites PNG de submarino e animais, para melhorar a apresentação visual.	Alta	3
US14	Como jogador, quero telas de game over, vitória e pontuação final, para visualizar o resultado da partida.	Alta	3
US15	Como jogador, quero efeitos visuais de profundidade e partículas de água, para aumentar a imersão no ambiente subaquático.	Média	3
US16	Como jogador, quero que bugs de desconexão e reconexão sejam tratados, para evitar travamentos durante a partida.	Alta	3
US17	Como equipe de desenvolvimento, queremos realizar testes finais, corrigir bugs críticos e gerar a versão de produção.	Alta	3
US35	Como jogador, quero fazer login por apelido antes de entrar na partida, para que meu nome apareça no HUD e na pontuação.	Alta	3

ID	História de Usuário	Prioridade	Sprint
----	---------------------	------------	--------

Tabela 4 – Product Backlog do Projeto

5.2 Sprint Backlog – Sprint 1

Período: 21/05/2026 a 26/05/2026

Objetivo: construir a infraestrutura base do projeto, criando o monorepo, os primeiros microserviços, a cena inicial em Phaser, o mapa base e o deploy inicial dos serviços.

ID	Tarefa	Responsável	Estim.	Status
T01	Estruturar o monorepo e configurar o Docker Compose com os serviços do projeto.	Equipe Backend	12h	Concluído
T02	Criar a Puzzle API REST com catálogo de 5 animais e endpoints de <i>guess</i> e <i>hint</i> .	Equipe Backend	10h	Concluído
T03	Criar o bootstrap do Vite com Phaser, cenas iniciais e GameScene base.	Equipe Frontend	8h	Concluído
T04	Implementar o MovementSystem com interpolação e o PlayerSubmarine com sprite.	Equipe Frontend	6h	Concluído
T05	Implementar o MapGenerationSystem com seed, rochas colisoras e algas decorativas.	Equipe Frontend	10h	Concluído
T06	Configurar o deploy no Render dos serviços e frontend, com variáveis de ambiente e CORS.	Equipe DevOps	8h	Concluído

Tabela 5 – Sprint Backlog – Sprint 1

5.3 Sprint Backlog – Sprint 2

Período: 26/05/2026 a 31/05/2026

Objetivo: implementar as principais mecânicas do jogo, incluindo multiplayer em tempo real, HUD, interação com animais, puzzles, oxigênio, dano e condições de derrota.

ID	Tarefa	Responsável	Estim.	Status
T07	Implementar sincronização de posição via Socket.IO usando os eventos <code>player:move</code> e <code>player:moved</code> .	Equipe Backend/Frontend	14h	Concluído
T08	Criar componente HUD reutilizável com barra de O ₂ e corações de vida.	Equipe Frontend	8h	Concluído
T09	Implementar detecção de proximidade com animais, evento <code>animal:approach</code> e abertura de puzzle por <code>puzzle:start</code> .	Equipe Backend/Frontend	12h	Concluído
T10	Criar a PuzzleScene completa, com slots de letras, input, dica, fechamento e evento <code>puzzle:end</code> .	Equipe Frontend	18h	Concluído
T11	Implementar lógica de O ₂ , corações, respawn e eliminação definitiva no estado da partida.	Equipe Backend/Frontend	14h	Concluído
T12	Realizar teste de integração manual com partida completa de ponta a ponta.	Equipe DevOps	6h	Concluído

Tabela 6 – Sprint Backlog – Sprint 2

5.4 Sprint Backlog – Sprint 3

Período: 31/05/2026 a 01/06/2026

Objetivo: realizar o polimento do jogo, incluindo autenticação por apelido, sprites finais, tela de resultado, efeitos visuais, correções de conexão e preparação da versão de produção.

ID	Tarefa	Responsável	Estim.	Status
T13	Implementar o Auth Service, a NicknameScene e o middleware de autenticação no Game Service.	Equipe Backend/Frontend	16h	Concluído
T14	Integrar assets PNG de submarinos, cinco animais, corações e bolha de O ₂ via assetsMap.	Equipe DevOps/Frontend	8h	Concluído
T15	Implementar a EndScene com tabela de pontuação, resumo de jogadores e integração com <code>game:restart</code> .	Equipe Backend/Frontend	12h	Concluído

ID	Tarefa	Responsável	Estim.	Status
T16	Implementar o WaterEffectsSystem com bolhas no mapa e vinheta fixa na câmera.	Equipe Frontend	6h	Concluído
T17	Corrigir fluxo de desconexão, destruição da sala e retorno ao menu.	Equipe Backend	8h	Concluído
T18	Realizar testes finais, correção de bugs críticos e build de produção.	Equipe DevOps	10h	Em andamento

Tabela 7 – Sprint Backlog – Sprint 3

6 Arquitetura do sistema com microserviços

6.1 Arquitetura do sistema utilizando microserviços

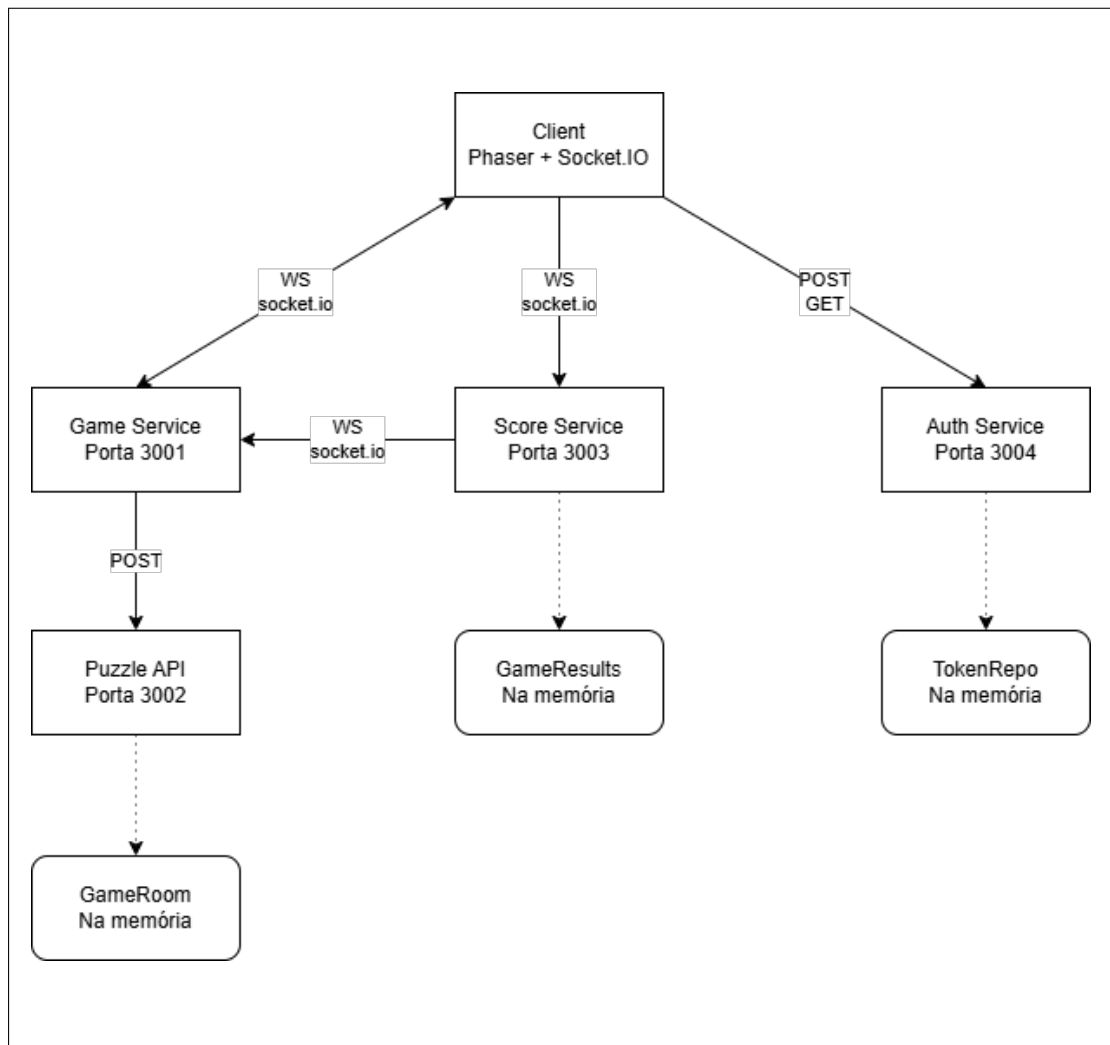


Figura 1 – Arquitetura do sistema

6.2 API Gateway

Neste projeto, optou-se por não implementar um API Gateway dedicado, pois o fluxo principal do jogo não é baseado apenas em requisições HTTP tradicionais, mas sim em comunicação bidirecional em tempo real. Como se trata de um jogo multiplayer online, eventos como movimentação dos submarinos, atualização de oxigênio, abertura de puzzles, colisões, respawn, fim de jogo e reinício da partida precisam ser enviados e recebidos continuamente entre frontend e backend.

Por esse motivo, foi utilizado o Socket.IO como principal mecanismo de comunicação entre o frontend e os serviços responsáveis pelo estado da partida. Diferente de uma requisição REST comum, em que o frontend envia uma solicitação e aguarda uma resposta pontual, o Socket.IO mantém uma conexão aberta entre cliente e servidor. Assim, o frontend pode emitir eventos para o backend, e o backend também pode enviar atualizações para os jogadores sempre que o estado da partida muda.

Assim, embora não exista um API Gateway centralizando todas as chamadas, existe uma separação clara de responsabilidades entre os microserviços. O frontend se comunica diretamente com os serviços necessários, enquanto o game-service atua como o principal coordenador da partida em tempo real. Essa decisão reduz complexidade adicional, evita uma camada intermediária que precisaria também suportar WebSocket, e mantém a arquitetura adequada ao tipo de aplicação desenvolvida.

Portanto, a ausência de API Gateway não compromete a arquitetura de microserviços do projeto. Ela representa uma decisão técnica compatível com o domínio do sistema: um jogo online que exige sincronização em tempo real. Em uma evolução futura, um API Gateway poderia ser adicionado para centralizar URLs públicas, autenticação e roteamento, mas ele não era essencial para o funcionamento da versão atual.

6.3 Especificação dos microserviços e componentes de software

6.3.1 Microserviço 1: [Auth Service — porta 3004]

Responsabilidade: Autenticar jogadores por apelido, emitir tokens JWT e validar tokens para outros microserviços.

Método	Rota	Descrição / Estrutura
POST	/api/login	Recebe: { nickname } Validação: 2–16 caracteres, alfanumérico e _ Retorna: { token, nickname }
GET	/api/validate/:token	Verifica se o token existe e não expirou. Retorna: { valid, nickname }

Tabela 8 – Especificação dos endpoints de autenticação

```
1
2 # Arquivo: auth-service/src/services/AuthService.ts
3 login(request: LoginRequest): LoginResponse {
```

```

4  const nickname = this.assertValidNickname(request.nickname);
5  const token = this.tokenRepo.save(nickname);
6  return { token, nickname };
7  }
8
9  validate(token: string): ValidateResponse {
10   const record = this.tokenRepo.findByToken(token);
11   if (!record) return { valid: false };
12   return { valid: true, nickname: record.nickname };
13 }

```

Listing 6.1 – Código do microserviço Auth Service

6.3.2 Microserviço 2: [Game Service — porta 3001]

Responsabilidade: Gerenciar toda a lógica de jogo em tempo real. Mantém o estado da sala (GameRoom) em memória, autentica jogadores via Auth Service e coordena puzzles com a Puzzle API.

Protocolo	Evento	Descrição / Carga Útil (Payload)
ws	game:start	Emitido quando a sala tem 2 jogadores. Inclui: <code>seed</code> , <code>playerIds</code> e <code>nicknames</code> .
ws	state:update	<i>Snapshot</i> autoritativo de todos os jogadores (<code>O₂</code> , vidas, posição) após cada evento.
ws	game:over	Encerra a partida. Retorna: <code>winner</code> , <code>reason</code> (exploration ou elimination), <code>discoveredAnimals</code> e <code>players</code> .
ws	player:move player:moved	<i>Frontend</i> → <i>backend</i>: envia posição local. <i>Backend</i> → <i>frontend</i>: faz <i>rebroadcast</i> para o parceiro.

Tabela 9 – Especificação dos eventos WebSocket do microserviço de Jogo

```

1
2 # Arquivo: game-service/src/game/GameRoom.ts

```

```

3 getDiscoveredAnimalsPayload(): DiscoveredAnimalRaw[] {
4   for (const animal of this.animals.values()) {
5     if (!animal.discovered) continue;
6     result.push({
7       animalId: animal.id,
8       discoveredBy: animal.discoveredBy ?? '',
9       timeToDiscoverMs: animal.getElapsedMs(),
10      wrongGuesses: animal.wrongGuesses,
11      pointsBase: catalogEntry?.points ?? 10,
12    });
13  }
14  return result;
15 }

```

Listing 6.2 – Código do microserviço Game Service

6.3.3 Microserviço 3: [Puzzle API — porta 3002]

Responsabilidade: Gerenciar o catálogo de animais e validar as interações relacionadas aos puzzles (adivinhação e dicas) durante a partida. Serviço stateless, ou seja não mantém estado de sessão. Nunca acessado diretamente pelo frontend.

Método	Rota	Descrição / Estrutura
GET	/api/animals	Retorna a lista de animais com os nomes mascarados (<code>hiddenName</code>). Utilizado pelo <i>Game Service</i> para a inicialização da sala.
POST	/api/puzzle/guess	Valida uma tentativa de letra para o animal. Recebe: { <code>animalId</code> , <code>letter</code> } Retorna: { <code>correct</code> , <code>positions[]</code> } contendo as posições reveladas caso a letra exista.
POST	/api/puzzle/hint	Solicita uma dica específica para o puzzle. Recebe: { <code>animalId</code> , <code>hintIndex</code> }

Método	Rota	Descrição / Estrutura
		Retorna: { hint } contendo a dica correspondente ao índice informado.

Tabela 10 – Especificação dos endpoints da API de puzzles

```

1
2 # Arquivo: puzzle-api\src\services\PuzzleService.ts
3 guess(animalId: string, letter: string): GuessResult | null {
4     const animal = this.repo.findById(animalId);
5     if (!animal) return null;
6
7     const positions: number[] = [];
8     const nameLower = this.normalizeLetter(animal.name);
9     const letterLower = this.normalizeLetter(letter);
10
11     for (let i = 0; i < nameLower.length; i++) {
12         if (nameLower[i] === letterLower) positions.push(i);
13     }
14
15     return { correct: positions.length > 0, positions };
16 }
17
18 hint(animalId: string, hintIndex: number): HintResult | null {
19     const animal = this.repo.findById(animalId);
20     if (!animal) return null;
21
22     return { hint: animal.hints[hintIndex] ?? 'Sem mais dicas.' };
23 }

```

Listing 6.3 – Código do microserviço Puzzle API

6.3.4 Microserviço 4: [Score Service — porta 3003]

Responsabilidade: Calcular e distribuir a pontuação final da partida. Conecta-se ao Game Service como cliente WebSocket interno (GameBridge), recebe o evento game:over, aplica as regras de pontuação e emite game:result ao frontend.

Regras de pontuação implementadas:

- Cada segundo abaixo de 60s vale +1 ponto de bônus de tempo (máximo de +30 pontos).
- Cada erro de letra penaliza −3 pontos, garantindo que a pontuação total nunca seja negativa.
- A fórmula para o cálculo da pontuação final é dada por:

$$totalPoints = \max(0, pointsBase + timeBonus - wrongGuesses \times 3)$$

Método/Prot.	Rota / Evento	Descrição / Estrutura
GET	/api/scores/latest	Retorna o resultado da última partida calculada.
GET	/api/scores	Lista todos os resultados de partidas armazenados em memória, ordenados por data decrescente.
WS	game:result	Evento emitido ao <i>frontend</i> contendo os campos: <code>winner</code> , <code>reason</code> , <code>animalScores[]</code> e <code>playerSummaries[]</code> .

Tabela 11 – Especificação dos endpoints e eventos do microserviço de pontuação

```

1
2 # Arquivo: score-service\src\socket\GameBridge.ts
3 export class GameBridge {
4   private client: ClientSocket | null = null;
5
6   constructor(
7     private readonly io: IoServer, // Server para o frontend
8     private readonly scoreService: ScoreService, // Calculadora de pontos
9     private readonly gameServiceUrl: string // URL do Game Service
10  ) {}
11
12  connect(): void {
13    // Conecta-se ao Game Service como cliente interno
14    this.client = ioClient(this.gameServiceUrl, {
15      auth: { clientType: 'service', serviceName: 'score-service' },
16    });
17
18    this.registerGameEvents();

```

```

19   }
20
21   private registerGameEvents(): void {
22       // Recebe o evento game:over do Game Service
23       this.client?.on('game:over', (payload: GameOverPayload) => {
24           try {
25               // Aplica as regras de pontuação
26               const result = this.scoreService.processGameOver(payload);
27
28               // Emite o game:result calculado para o frontend
29               this.io.emit('game:result', result);
30
31           } catch (err) {
32               console.error('[GameBridge] erro ao processar game:over:', err);
33           }
34       });
35   }
36 }

```

Listing 6.4 – Integração do Score Service com o Game Service via GameBridge

6.3.5 Componentes de software: Frontend

Componente / Sistema	Descrição e responsabilidades
SocketManager	Componente central de comunicação com o <i>game-service</i> . Encapsula Socket.IO, tipa os eventos de <i>gameplay</i> e mantém o último <i>state:update</i> recebido.
ScoreSocketManager	Componente de integração com o <i>score-service</i> . Escuta o evento <i>game:result</i> , que entrega ao <i>frontend</i> o resultado final calculado pelo <i>backend</i> .
NicknameScene	Componente de login. Envia o <i>nickname</i> ao <i>auth-service</i> via HTTP, armazena o token/ <i>nickname</i> de sessão e só libera o fluxo para o Socket.IO quando o login é aceito.

Componente / Sistema	Descrição e responsabilidades
MenuScene	Componente de entrada e <i>matchmaking</i> visual. Exibe a tela de espera pelo segundo jogador e inicia a partida quando recebe o evento <code>game:start</code> .
GameScene	Componente principal do jogo. Cria o mapa, submarinos, câmera, HUDs, animais e integra os eventos de <i>multiplayer</i> . Apenas congela/transiciona o fim da partida, sem calcular pontuações.
PuzzleScene	Componente do minigame da força. Recebe os dados do animal e envia tentativas de letras e requisições de dicas ao <i>backend</i> .
EndScene	Componente visual de resultado final. Exibe o vencedor, o motivo do término, animais descobertos, pontuação por animal, resumo por jogador e o botão “Jogar novamente”.
HUD	Componente de interface para exibição de O ₂ e vidas. A <code>GameScene</code> instancia uma HUD local e outra correspondente ao parceiro.
PlayerSubmarine	Componente ou entidade visual do jogador. Representa tanto o submarino controlado localmente quanto o parceiro remoto.
Animal	Componente ou entidade de interação. Possui um raio de detecção utilizado para emitir sinais de aproximação do jogador.
MovementSystem	Componente de lógica de movimento local. Gerencia a velocidade, a direção visual e realiza os cálculos baseados no <i>delta time</i> de forma isolada da <code>GameScene</code> .
MapGenerationSystem	Componente de geração do mapa. Utiliza uma <i>seed</i> (semente numérica) para criar de forma consistente a grade, os obstáculos e as decorações do cenário.

Componente / Sistema	Descrição e responsabilidades
WaterEffectsSystem	Componente visual atmosférico. Adiciona bolhas e partículas de água no mundo, além de uma vinheta fixa na câmera, atuando puramente na estética, sem participar das regras do jogo.

Tabela 12 – Descrição dos componentes do sistema *frontend*

6.3.6 Componentes de software: Backend

Componente / Sistema	Descrição e responsabilidades
AuthServer	Servidor Express que configura CORS, <i>middlewares</i> e monta as rotas de <i>login</i> e validação de <i>token</i> . Ponto de entrada do microsserviço de autenticação.
AuthController	Camada de controle HTTP. Expõe POST <code>/api/login</code> e GET <code>/api/validate/:token</code> . Mapeia erros de validação para respostas HTTP 400/401/500 sem vaziar detalhes internos.
AuthService	Regras de negócio de autenticação. Valida o apelido (2–16 chars, alfanumérico+_), delega a criação do <i>token</i> ao <code>TokenRepository</code> e retorna o par <code>{ token, nickname }</code> . Lança <code>AuthValidationError</code> tipado para erros de validação.
TokenRepository	Persistência <i>in-memory</i> de <i>tokens</i> com TTL de 20 minutos. Usa <code>Map<string, TokenRecord></code> . Gera <i>tokens</i> via <code>crypto.randomUUID()</code> . Remove automaticamente <i>tokens</i> expirados na busca. Expõe <code>count()</code> para o <i>health check</i> .
AuthDTO	Contratos de dados do serviço: <code>LoginRequest</code> , <code>LoginResponse</code> , <code>ValidateResponse</code> e <code>TokenRecord</code> . Garante tipagem estrita entre camadas sem acoplamento concreto.

Componente / Sistema	Descrição e responsabilidades
GameServer	Servidor Socket.IO principal. Gerencia o ciclo de vida das conexões, executa o <i>middleware</i> de autenticação (valida <i>token</i> via <i>Auth Service</i>), faz <i>matchmaking</i> , inicia partidas, registra eventos de movimento, <i>puzzle</i> , colisão, <i>respawn</i> e <i>game over</i> . Mantém <i>cache</i> local de <i>tokens</i> validados (TTL 5 min) para reduzir chamadas HTTP ao <i>Auth Service</i> .
GameRoom	Estado central da partida em andamento. Gerencia o mapa de jogadores (PlayerState) e de animais (AnimalState), controla o <i>puzzle</i> ativo, gera <i>seeds</i> aleatórias, fornece o <i>payload</i> de animais descobertos para o <i>Score Service</i> e determina o jogador líder por contagem de descobertas.
PlayerState	Modelo de estado de um jogador: posição (x, y), vidas (3 corações), oxigênio (100%), contador de mortes e <i>nickname</i> . Métodos isDead() , respawn() e reset() encapsulam as transições de estado sem expor lógica ao GameServer .
AnimalState	Modelo de estado de um animal no mapa: posição, <i>flag</i> de descoberta, <i>timestamp</i> de início do <i>puzzle</i> , contador de erros e id do descobridor. Métodos startPuzzle() , registerWrongGuess() e getElapsedMs() rastreiam os dados necessários para o cálculo de pontuação.
SocketEvents	Enumeração centralizada de todos os nomes de eventos Socket.IO do sistema (game:start , state:update , puzzle:start , game:over , etc.). Garante consistência e evita <i>strings</i> mágicas dispersas no código.

Componente / Sistema	Descrição e responsabilidades
App (PuzzleAPI)	Servidor Express do microserviço de <i>puzzles</i> . Instancia e compõe AnimalRepository , PuzzleService e PuzzleController via injeção de dependência manual (IoC). Único ponto de entrada e configuração da <i>Puzzle API</i> .
PuzzleController	Controla os três <i>endpoints</i> REST da <i>Puzzle API</i> : GET <code>/api/animals</code> (lista com nomes mascarados), POST <code>/api/puzzle/guess</code> (valida letra) e POST <code>/api/puzzle/hint</code> (retorna dica). Valida entrada e delega ao PuzzleService .
PuzzleService	Lógica do minigame de forca. Normaliza letras removendo acentos antes da comparação (ex.: "ã" → "a"), encontra posições da letra no nome do animal e retorna as posições reveladas. Gera os nomes mascarados com <i>underscores</i> preservando hífens e espaços.
AnimalRepository	Fonte de dados dos animais. Carrega o catálogo estático (animals.ts) em memória na inicialização e expõe findAll() e findById() . Isola o restante da aplicação do formato de armazenamento dos dados.
ScoreServer	Servidor Express + Socket.IO do microserviço de pontuação. Compõe todas as dependências via IoC manual, configura CORS, monta as rotas REST de <i>scores</i> , inicializa os <i>handlers</i> Socket.IO para o <i>frontend</i> e inicia a conexão do GameBridge ao <i>Game Service</i> na subida do serviço.

Componente / Sistema	Descrição e responsabilidades
GameBridge	Cliente Socket.IO interno que conecta o <i>Score Service</i> ao <i>Game Service</i> . Escuta o evento <code>game:over</code> com deduplicação por <i>cooldown</i> de 3s, repassa o <i>payload</i> ao <code>ScoreService</code> para cálculo e emite <code>game:result</code> ao <i>frontend</i> . Gerencia reconexão automática com <i>backoff</i> exponencial e <i>replay</i> do último resultado para clientes que reconectam.
ScoreService	Orquestrador do cálculo de pontuação. Normaliza o <i>payload</i> recebido do <i>Game Service</i> (suportando variações de formato), delega o cálculo ao <code>ScoreCalculator</code> , agrega os resumos por jogador, inclui jogadores sem descobertas com pontuação zero e persiste o resultado no <code>GameResultRepository</code> .
ScoreCalculator	Algoritmo puro de pontuação. Calcula bônus de tempo (cada segundo abaixo de 60s vale +1 pt, máx. +30) e penalidade por erro (−3 pts por chute errado, total nunca negativo). Método <code>calculatePlayerSummaries()</code> agrega pontos por jogador e ordena por pontuação decrescente.
ScoreController	Expõe os três <i>endpoints</i> REST do <i>Score Service</i> : <code>GET /api/scores</code> (todos os resultados), <code>GET /api/scores/latest</code> (último resultado, retorna 200 com <i>null</i> se vazio) e <code>GET /api/scores/:id</code> (resultado por id, 404 se não encontrado).
GameResultRepository	Persistência <i>in-memory</i> dos resultados de partidas. Usa <code>Map<string, GameResult></code> . Ordena por data decrescente em <code>findAll()</code> . O id de cada resultado é gerado como <code>timestamp-random</code> para garantir unicidade sem dependência externa.

Componente / Sistema	Descrição e responsabilidades
GameResult	Modelo imutável do resultado de uma partida. Armazena vencedor, motivo do encerramento, lista de pontuações por animal (<code>AnimalScoreEntry[]</code>) e resumo por jogador (<code>PlayerScoreSummary[]</code>). Gerado pelo <code>ScoreService</code> e persistido pelo <code>GameResultRepository</code> .
ScoreDTO	Contratos de dados de pontuação: <code>GameOverPayload</code> (recebido do <i>Game Service</i>), <code>DiscoveredAnimalRaw</code> , <code>AnimalScoreEntry</code> , <code>PlayerScoreSummary</code> e <code>GameResultPayload</code> (emitido ao <i>frontend</i>). Suporta variações de campo para compatibilidade com versões do <i>Game Service</i> .

Tabela 13 – Descrição detalhada dos componentes de *software* do *backend*

6.4 Sistema de recomendação

O projeto não implementa um sistema de recomendação tradicional (baseado em perfil ou histórico), uma vez que o escopo central da aplicação é validar uma arquitetura distribuída de microsserviços e comunicação em tempo real. Contudo, o sistema incorpora mecanismos de suporte e *feedback* integrados à sua arquitetura para enriquecer a experiência do jogador.

Durante as sessões de jogo, a *Puzzle API* atua como um sistema de assistência reativa, fornecendo dicas textuais progressivas que guiam os jogadores na resolução do minigame sem expor a resposta direta. Adicionalmente, o *Score Service* realiza a avaliação de desempenho, entregando um diagnóstico personalizado ao final da partida com base em métricas de tempo, precisão e etc.

Dessa forma, a arquitetura substitui a recomendação preditiva por um modelo de auxílio heurístico e gamificação estruturada.

6.5 Microservios em execuo

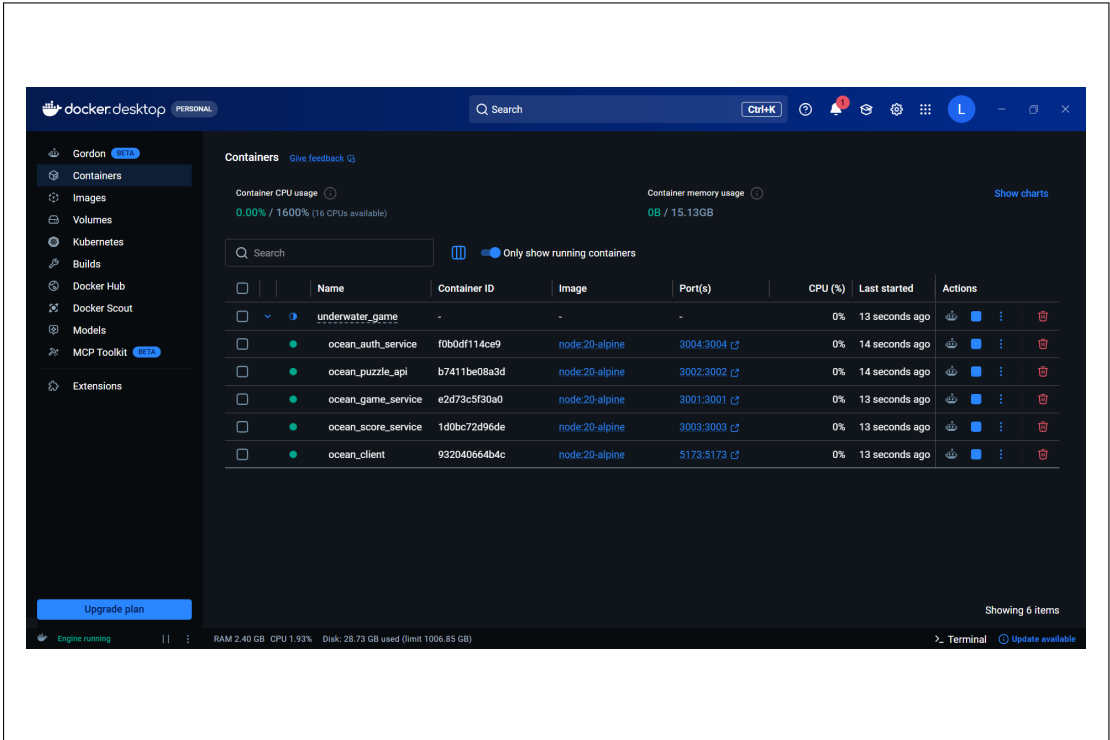


Figura 2 – Listagem de todos os microservios em execuo

7 Tecnologias utilizadas

Microserviço	Linguagem	Tecnologias	Banco de Dados
API Gateway	-	-	-
Auth Service	TypeScript	HTTP, Node.js e UUID (crypto)	Não utiliza
Game Service	TypeScript	Node.js e Socket.IO	Não utiliza
Puzzle API	TypeScript	HTTP e Node.js	Não utiliza
Score Service	TypeScript	HTTP, Node.js e Socket.IO	Não utiliza
Recomendação	-	-	-

Tabela 14 – Tecnologias utilizadas por microserviço

Descrição e justificativa de tecnologias

TypeScript: É um superconjunto do JavaScript que adiciona tipagem estática opcional à linguagem. Foi escolhido para compor todos os microserviços do sistema visando garantir maior segurança estrutural no desenvolvimento. A tipagem estática previne erros de execução (runtime errors) e facilita a criação de contratos de dados estritos (DTOs e interfaces), garantindo que as informações trafegadas entre o frontend e os diversos microserviços mantenham uma estrutura consistente.

Node.js: Consiste em um ambiente de execução JavaScript server-side, caracterizado por sua arquitetura orientada a eventos e modelo de entrada/saída (I/O) não bloqueante. Sua adoção justifica-se pela alta eficiência em lidar com múltiplas conexões concorrentes, um requisito fundamental para suportar as requisições simultâneas de um jogo multiplayer. Além disso, permitiu a padronização do ecossistema, utilizando a mesma base de linguagem (JavaScript/TypeScript) tanto no cliente quanto no servidor.

Socket.IO: É uma biblioteca que abstrai e gerencia conexões WebSocket, fornecendo uma comunicação bidirecional, orientada a eventos e de baixíssima latência entre clientes e servidores. Foi a tecnologia escolhida para fundamentar o Game Service e o Score Service, pois um jogo competitivo exige a sincronização em tempo real de estados críticos (como movimentação espacial, coleta de itens e consumo de O2) de forma contínua e com o menor atraso (delay) possível, algo inviável em requisições HTTP tradicionais.

HTTP: O protocolo padrão de transferência de hipertexto atua como a base da

comunicação para operações pontuais do sistema. Foi mantido e escolhido para rotas específicas do Auth Service e da Puzzle API devido à natureza stateless (sem estado) dessas interações. Como validações de apelido (login), requisições de dicas e validações de letras na força são eventos de requisição-resposta diretos que não demandam um canal aberto contínuo, o HTTP se mostra a abordagem mais leve e adequada.

UUID (via módulo nativo Crypto): Representa o padrão de identificadores únicos universais (Universally Unique Identifiers). Foi adotado no Auth Service para a geração de tokens de sessão. A escolha se deve à necessidade de criar chaves de acesso alfanuméricas descentralizadas, temporárias e de alta entropia (seguras e sem risco de colisão), permitindo a autenticação dos jogadores em memória sem a obrigatoriedade e o custo de latência de um banco de dados persistente.

8 Telas do sistema

8.1 Tela de login

A interface de acesso inicial ao jogo (figura 3) consiste na tela de identificação do usuário. Desenvolvida de forma minimalista, a tela apresenta o título do sistema e um campo de entrada de texto destinado à inserção do nickname (apelido) do jogador.

Ao preencher o campo e acionar o botão "Entrar", o client (desenvolvido com Phaser e Socket.IO) inicia o fluxo de autenticação. O frontend realiza uma requisição HTTP para o microsserviço Auth Service, que valida as regras do apelido informado e, em caso de sucesso, gera um token de acesso temporário. Esse token é armazenado localmente para a sessão, permitindo que o jogador avance para a próxima tela do sistema.

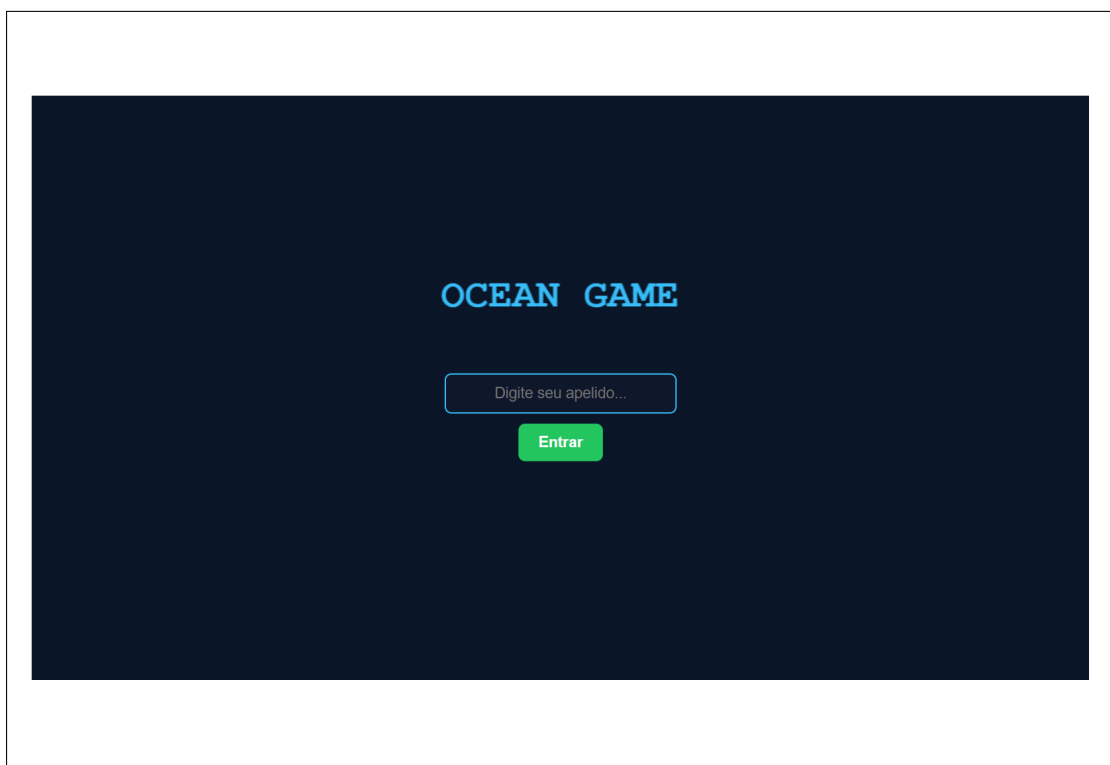


Figura 3 – Tela de login

8.2 Esperando por um segundo jogador

Após a autenticação bem-sucedida, o usuário é direcionado para a tela de espera, conforme ilustrado na figura 4. Essa interface tem o objetivo principal de fornecer feedback visual ao jogador: ela confirma que o sistema o reconheceu, exibindo uma saudação personalizada com o seu nickname, e informa que a busca por uma partida está em

andamento.

Sob a perspectiva do fluxo do sistema, este é o momento em que o client estabelece a comunicação em tempo real via WebSocket com o microsserviço gerenciador da partida (Game Service). O jogador é colocado em um lobby (sala de aguardo) e ali permanece até que um segundo usuário se conecte. Assim que a capacidade exigida pela sala é atingida (dois jogadores), o servidor emite o evento de início da partida, e a interface transiciona automaticamente para o cenário principal do jogo, sem exigir nova interação do usuário.



Figura 4 – Esperando o segundo jogador

8.3 Cenário principal do jogo

A figura 5 apresenta a interface principal onde a mecânica central do jogo se desenrola. O ambiente virtual é renderizado em uma perspectiva bidimensional top-down (visto de cima), exibindo um mapa em grade pelo qual os submarinos dos jogadores podem explorar e interagir com a fauna marinha disposta pelo cenário.

Um elemento fundamental desta tela é o Heads-Up Display (HUD), posicionado nos cantos inferiores. O HUD exibe o estado em tempo real de ambos os jogadores de forma simultânea, contendo seus respectivos nicknames, barras de oxigênio (que atuam como a restrição de tempo da partida) e medidores de vida (representados por ícones de coração). Essa exibição espelhada é crucial para o design competitivo do jogo, pois permite que um jogador monitore os recursos críticos do seu adversário e planeje suas ações de acordo.

Sob o aspecto técnico, essa tela reflete o estado em memória mantido pelo microsserviço

Game Service. Cada movimento realizado localmente é enviado via WebSocket para o servidor, que processa a ação e realiza o rebroadcast para o parceiro, garantindo que as posições dos submarinos, os níveis de oxigênio e as interações com os animais permaneçam perfeitamente sincronizadas em ambos os clients.

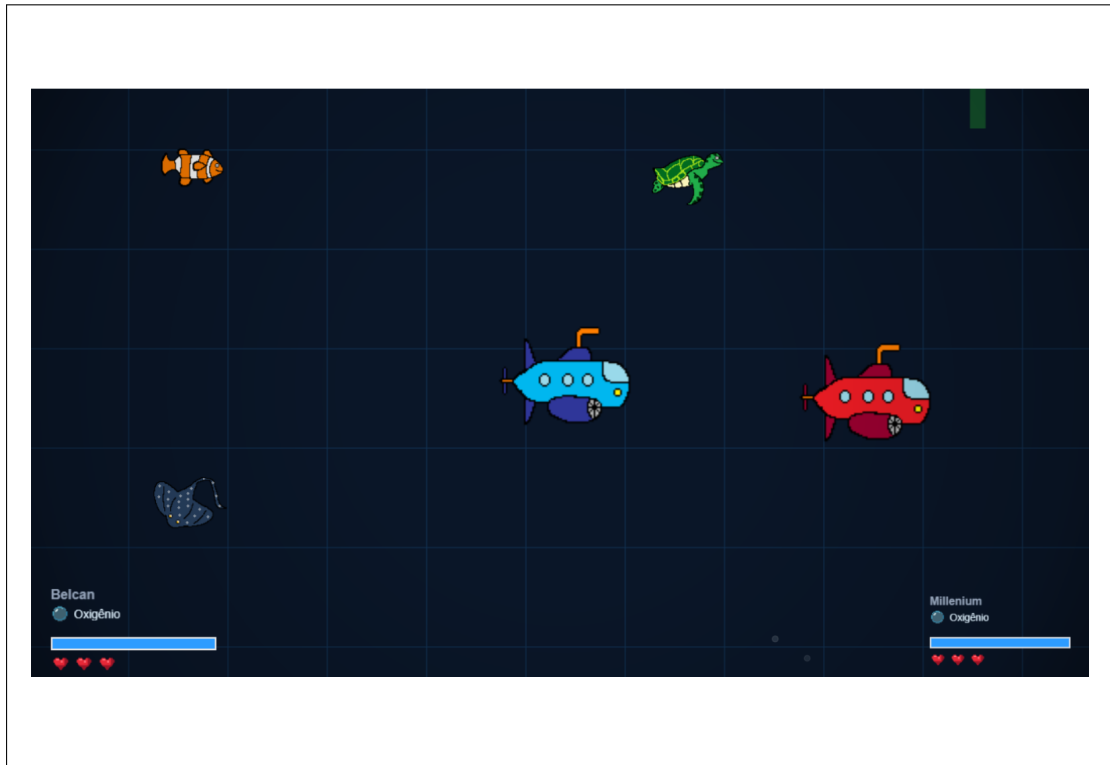


Figura 5 – Mapa e jogadores conectados

8.4 Interface do puzzle

A figura 6 ilustra a interface apresentada aos jogadores ao interagirem com um dos animais marinhos espalhados pelo mapa. O sistema exibe uma janela de sobreposição (overlay), ofuscando o cenário principal para dar foco ao minigame denominado "Forca submarina".

Esta interface é composta por elementos interativos projetados para o desafio cognitivo: ao centro, o nome do animal oculto representado por traços; um campo de entrada para a submissão de letras; e um botão para acionar o sistema progressivo de dicas textuais. Adicionalmente, a tela exibe um monitoramento isolado de oxigênio, integrando a restrição de recursos do jogo à resolução do quebra-cabeça.

Do ponto de vista da arquitetura de software, as ações realizadas nesta tela disparam eventos diretos para o Puzzle API. Cada letra digitada ou dica solicitada gera uma requisição ao backend, que processa a tentativa e devolve as posições corretas ou o texto da dica. Essa abordagem stateless e validada no servidor garante a segurança e a integridade da partida, impedindo que a resposta do puzzle fique exposta no código-fonte do navegador

do cliente antes de ser efetivamente descoberta.

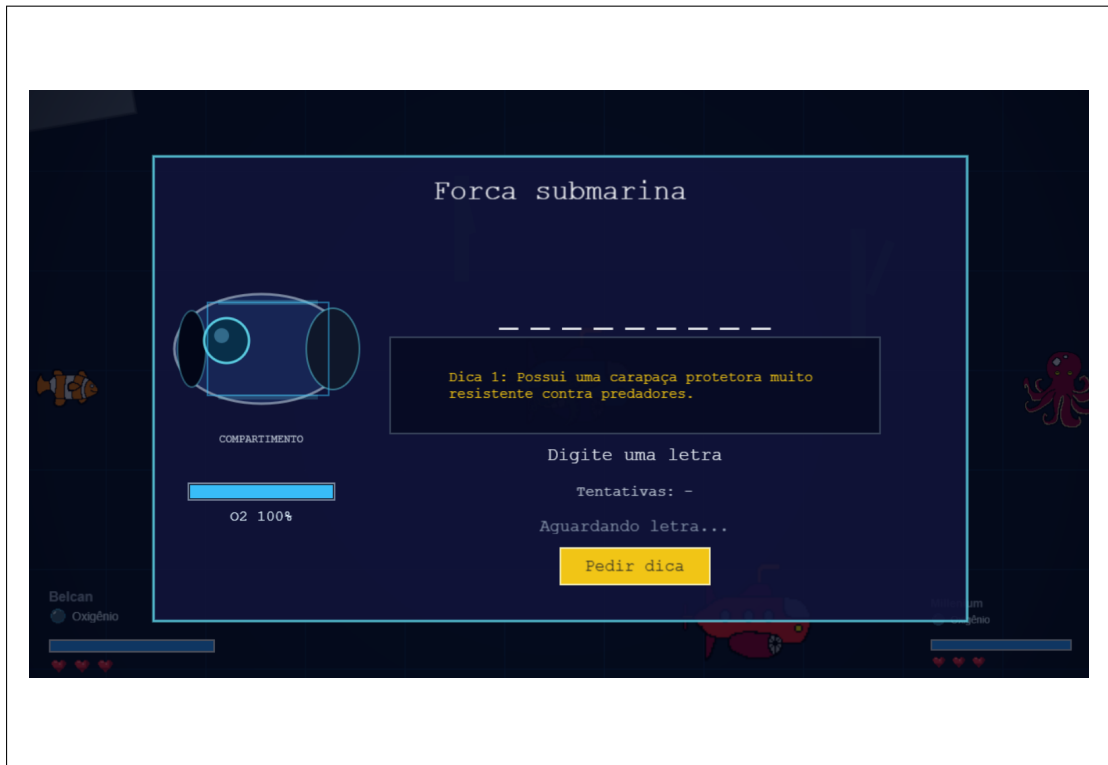


Figura 6 – Tela de puzzle

8.5 Interface de resultados da partida

A figura 7 apresenta a interface de conclusão (componente EndScene), exibida automaticamente ao término da sessão de jogo. Assim como a tela do minigame, esta interface sobrepõe o cenário principal, congelando a ação para fornecer o feedback analítico final aos usuários. A tela adapta dinamicamente seu título de acordo com o motivo do encerramento da partida, como o sucesso total ("Exploração concluída!") ou falhas (como esgotamento de oxigênio).

Visualmente, a tela é estruturada em painéis informativos para detalhar tanto o aspecto cooperativo quanto o competitivo do jogo. À esquerda, exibe-se o resultado geral com a declaração do vencedor e um resumo consolidado contendo a pontuação final e a quantidade de descobertas de cada jogador. À direita, uma tabela minuciosa destrincha a pontuação adquirida por animal, evidenciando a aplicação prática das regras de negócio do sistema: são exibidos os pontos base, os bônus de agilidade (tempo) e as penalidades aplicadas por erros.

Do ponto de vista arquitetural, é importante ressaltar que os valores apresentados não são calculados localmente. O frontend atua de forma passiva neste estágio, limitando-se a escutar o evento `game:result` transmitido de forma assíncrona pelo Score Service. Ao receber o payload final validado pelo servidor, a interface renderiza as estatísticas e disponibiliza o

botão interativo "Jogar novamente", que reinicia o ciclo de vida do client e o redireciona ao fluxo de matchmaking.



Figura 7 – Tela fim de jogo

9 Repositório no GitHub

O código-fonte completo do projeto está disponível no GitHub no seguinte endereço:

https://github.com/LarissaFDS/underwater_game.git

9.1 Instruções para execução do sistema

Esta seção apresenta o passo a passo para executar o *Underwater Game* em ambiente local. A forma recomendada de execução é utilizando Docker Compose, pois todos os microserviços e o frontend são inicializados de maneira integrada.

9.1.1 Pré-requisitos

Para executar o projeto localmente, é necessário possuir os seguintes recursos instalados:

- Git;
- Docker e Docker Compose;
- Navegador web atualizado;
- Node.js 18 ou superior, apenas caso seja necessário executar os serviços manualmente, sem Docker.

9.1.2 Execução com Docker Compose

1. Clone o repositório:

```
1 git clone https://github.com/LarissaFDS/underwater_game.git
2 cd underwater_game
```

2. Inicie todos os serviços com Docker Compose:

```
1 docker compose up --build
```

3. Aguarde a inicialização dos containers. Ao final do processo, o frontend estará disponível no navegador em:

```
1 http://localhost:5173
```

4. Para testar o modo multiplayer, abra o jogo em duas abas, dois navegadores diferentes ou duas máquinas conectadas ao mesmo ambiente. Cada jogador deve informar um apelido na tela inicial. A partida só começa quando dois jogadores entram na sala.
5. Para encerrar a execução dos serviços, utilize:

```
1 docker compose down
```

9.1.3 Serviços executados localmente

Ao utilizar Docker Compose, os principais serviços do sistema são executados nas seguintes portas:

Serviço	URL local	Responsabilidade
Frontend	http://localhost:5173	Interface web do jogo, desenvolvida com Phaser e TypeScript.
Auth Service	http://localhost:3004	Realiza o login por apelido e valida o token do jogador.
Game Service	http://localhost:3001	Gerencia a sala multiplayer, estado dos jogadores e eventos em tempo real via Socket.IO.
Puzzle API	http://localhost:3002	Fornece os dados dos animais, dicas e validação dos puzzles.
Score Service	http://localhost:3003	Calcula e emite a pontuação final da partida.

Tabela 15 – Serviços locais do sistema

9.1.4 Execução manual sem Docker

Também é possível executar cada serviço manualmente, em terminais separados. Essa opção é útil para desenvolvimento e depuração individual dos microserviços.

A ordem recomendada de inicialização é:

1. `auth-service`;
2. `puzzle-api`;
3. `game-service`;

4. `score-service`;
5. `client`.

Para cada serviço, acesse sua pasta, instale as dependências e execute o modo de desenvolvimento:

```
1 cd auth-service
2 npm install
3 npm run dev
```

```
1 cd puzzle-api
2 npm install
3 npm run dev
```

```
1 cd game-service
2 npm install
3 npm run dev
```

```
1 cd score-service
2 npm install
3 npm run dev
```

```
1 cd client
2 npm install
3 npm run dev
```

No modo manual, o frontend deve estar configurado para acessar os serviços locais:

```
1 VITE_AUTH_URL=http://localhost:3004
2 VITE_SOCKET_URL=http://localhost:3001
3 VITE_SCORE_URL=http://localhost:3003
```

Como o frontend utiliza Vite, as variáveis iniciadas com `VITE_` precisam estar configuradas antes da execução do build.

9.1.5 Observações sobre execução

Durante a execução, o fluxo esperado é o seguinte:

- o jogador informa um apelido na tela inicial;
- o frontend realiza login no `auth-service`;

- após o login, o frontend conecta ao **game-service** via Socket.IO;
- o jogo aguarda dois jogadores para iniciar a partida;
- durante a partida, o **game-service** sincroniza os jogadores em tempo real;
- ao final, o **score-service** envia o resultado para a tela final.

Caso seja necessário limpar containers antigos ou recriar o ambiente local, pode-se utilizar:

```
1 docker compose down --remove-orphans -v
2 docker compose up --build --force-recreate
```

10 Vídeo do sistema

O vídeo demonstrativo do funcionamento do sistema está disponível no seguinte endereço:

https://drive.google.com/file/d/1kaGGiPsMdLR__N8zGRz0dgbKi87pxIv_/view?usp=drive_link

O vídeo demonstrativo apresenta o funcionamento geral do *Underwater Game*, desde a tela inicial de login até a tela de resultados da partida. Inicialmente, cada jogador informa um apelido válido e aguarda a entrada do segundo participante para que a partida seja iniciada. Em seguida, o vídeo mostra o ambiente principal do jogo, composto por submarinos, animais marinhos, obstáculos, barra de oxigênio e indicadores de vida.

Durante a demonstração, os jogadores exploram o mapa, desviam de obstáculos e interagem com os animais para iniciar os puzzles. Ao se aproximar de um animal, o jogador recebe uma dica textual e deve tentar descobrir seu nome no minigame de força submarina. O vídeo também evidencia a competição em tempo real, pois os dois jogadores podem disputar a descoberta dos animais enquanto acompanham o estado um do outro pela interface.

Também são apresentadas as principais penalidades do jogo, como perda de vida ao colidir com obstáculos e redução de oxigênio ao errar letras durante o puzzle. Caso o jogador perca suas chances, ocorre o fim da partida por eliminação. Ao final, a tela de resultados exibe o vencedor, a quantidade de animais descobertos por cada jogador e a pontuação final, considerando bônus de tempo e penalidades por erros.

11 Referências bibliográficas

- NEWMAN, Sam. **Building Microservices: Designing Fine-Grained Systems**. 2. ed. Sebastopol: O'Reilly Media, 2021.
- FOWLER, Martin; LEWIS, James. **Microservices**. Disponível em: <https://martinfowler.com/articles/microservices.html>. Acesso em: 21/05/2026.
- PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.
- SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2019.